

Sync vs Async in Python: A Detailed Guide

Introduction

In programming, especially in Python, understanding the difference between synchronous (sync) and asynchronous (async) execution is crucial. These concepts determine how your code handles tasks, particularly when some tasks take time to complete, like waiting for data from the internet or processing delays.

- **Synchronous Execution:** This is like doing tasks one after another in a line. You wait for the first task to finish completely before starting the next one.
- **Asynchronous Execution:** This allows tasks to run in parallel or overlap. You don't wait idly; instead, you start other tasks while waiting for the slow ones to finish.

We'll explain both in simple terms, with examples, and show how they apply to real-world scenarios like building agents in Python.

What is Synchronous Execution?

Synchronous code runs step by step. Each task must complete before the next one begins. This is straightforward but can be inefficient if a task takes a long time, as the program just waits.

When to Use Sync?

- For small, quick tasks.
- In simple projects where waiting isn't a big issue.
- When the order of tasks is critical, and no parallel work is needed.

Example of Sync in Python

Let's create a file called `testing.py` with this code:

```
# Sync Example: Tasks run step by step

import time

# Step 1: Simulate a test starting and taking 5 seconds
print("Test starting...")
time.sleep(5) # Wait for 5 seconds
print("Test completed.")

# Step 2: Simulate getting results, taking 3 seconds
print("Getting results...")
time.sleep(3) # Wait for 3 seconds
print("Results received.")
```

How It Works:

1. "Test starting..." prints immediately.
2. The program waits 5 seconds (using `time.sleep(5)`).
3. "Test completed." prints after the wait.
4. "Getting results..." prints right after, since there's no wait between these prints.
5. The program waits 3 seconds.
6. "Results received." prints.

When you run this (`uv run testing.py`), the output is sequential:

```
Test starting...
(5-second wait)
Test completed.
Getting results...
(3-second wait)
Results received.
```

The total time is about 8 seconds because it waits for each delay. This is sync: no overlapping, just waiting in line.

What is Asynchronous Execution?

Asynchronous code doesn't wait idly. If one task is slow, it starts others in the meantime. Tasks can run concurrently, saving time. Python uses the `asyncio` library for this.

When to Use Async?

- For tasks involving waiting, like API calls, internet requests, or file I/O.
- In larger projects where efficiency matters.
- When you have multiple tasks that can overlap without depending on each other strictly.

Key Difference from Sync

- **Sync:** Waits for each task to finish before moving on. It's like a single queue.
- **Async:** Continues with other tasks while waiting. It's like multitasking – background processes handle waits.

Example of Async in Python

Using `asyncio`, we make the code async. Create or update `testing.py`:

```
# Async Example: Tasks can overlap

import asyncio

async def start_test():
    print("Test starting...")
    await asyncio.sleep(5) # Simulate 5-second wait, but allow other tasks
    print("Test completed.")

async def get_results():
    print("Getting results...")
    await asyncio.sleep(3) # Simulate 3-second wait, but allow other tasks
    print("Results received.")

async def main():
    await asyncio.gather(start_test(), get_results()) # Run both tasks together

asyncio.run(main())
```

How It Works:

- We define async functions using ``async def``.
- ``await`` tells Python: "Wait here if needed, but let other tasks run."
- ``asyncio.gather()`` runs multiple tasks concurrently.
- ``asyncio.run(main())`` starts the async program.

```
Test starting...
Getting results...
(3-second wait passes first)
Results received.
(2 more seconds, total 5 for test)
Test completed.
```

Total time is about 5 seconds (the longest wait), not 8. The 3-second task overlaps with the 5-second one.

Switching Timings

Change the sleeps (test: 3s, results: 5s):

```
import asyncio

async def start_test():
    print("Test starting...")
    await asyncio.sleep(3)
    print("Test completed.")

async def get_results():
    print("Getting results...")
    await asyncio.sleep(5)
    print("Results received.")

async def main():
    await asyncio.gather(start_test(), get_results())

asyncio.run(main())
```

Output:

```
Test starting...
Getting results...
(3-second wait)
Test completed.
(2 more seconds)
Results received.
```

The order depends on which finishes first, but no time is wasted waiting idly.

Applying Sync vs Async to Agents

In advanced Python setups, like building AI agents (e.g., using libraries for chat completions), you can choose sync or async.

Sync Agent Example

Assume you have a setup with an `Agent` and `Runner` (from a framework like the one in the code). Here's a sync version in `mainandsync.py`:

```
from dotenv import load_dotenv
import os
from agents import Agent, Runner, OpenAI, OpenAIChatCompletionsModel, RunConfig

load_dotenv()
gemini_api_key = os.getenv("GEMINI_API_KEY")

if not gemini_api_key:
    raise ValueError("GEMINI_API_KEY is not set.")

external_client = OpenAI(
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)

model = OpenAIChatCompletionsModel(
    model="gemini-2.0-flash",
    openai_client=external_client
)

config = RunConfig(
    model=model,
    model_provider=external_client,
    tracing_disabled=True
)

agent = Agent(
    name="Translator",
    instructions="You are a helpful assistant who always translates Urdu sentences into clear and simple English."
)

response = Runner.run_sync(
    agent,
    input="'میرا نام ساحر احمد شیخ ہے۔ میں جی ائی اے اے ائی سی کا طالب علم ہوں۔',",
    run_config=config
)

print(response)
```

This runs synchronously: steps happen one by one.

Async Agent Example

To make it async, import ``asyncio``, use ``AsyncOpenAI``, and wrap in an async function. In ``appandasync.py``:

```
from dotenv import load_dotenv
import os
from agents import Agent, Runner, AsyncOpenAI, OpenAIChatCompletionsModel, RunConfig
import asyncio

load_dotenv()
gemini_api_key = os.getenv("GEMINI_API_KEY")

if not gemini_api_key:
    raise ValueError("GEMINI_API_KEY is not set.")

external_client = AsyncOpenAI(
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)

model = OpenAIChatCompletionsModel(
    model="gemini-2.0-flash",
    openai_client=external_client
)

config = RunConfig(
    model=model,
    model_provider=external_client,
    tracing_disabled=True
)

async def main():
    agent = Agent(
        name="Translator",
        instructions="You are a helpful assistant who always translates Urdu sentences into clear and simple English."
    )

    response = await Runner.run(
        agent,
        input='میرا نام سحر احمد شیخ ہے۔ میں جی ائی اے سی کا طالب علم ہوں۔',
        run_config=config
    )

    print(response)

asyncio.run(main())
```

Changes:

- Use ``AsyncOpenAI`` for async compatibility.
- Wrap in ``async def main()``.
- Use ``await Runner.run()`` for async execution.
- Start with ``asyncio.run(main())``.

This is efficient for agents handling waits (e.g., API calls).

Conclusion

- Sync: Simple, sequential, good for quick tasks. But it wastes time on waits.
- Async: Efficient, overlapping, ideal for I/O-bound tasks. Use ``asyncio`` to implement.

The complete agent code was covered in our previous PDF, which you can refer to in the "Step 00 to Step 05" document. In that PDF, I explained everything in detail. You can find it in the repository at:

<https://github.com/SahirAhmedSheikh814/GIAIC-SAT-Afternoon.git>

Within this repository, there is a folder named **Agentic AI Test Preparation**. Click on it to access all the PDFs, code, and other materials related to Agentic AI. I will cover all the steps from Sir Zia's repository and provide comprehensive preparation for the test, Inshallah.